BUILDEVCON · MARIADB / MYSQL ON KUBERNETES

Run and Operate MariaDB in a Cloud Native Way

Using the MariaDB Operator

Martin Montes · Lead Engineer, MariaDB Corporation

Martin Montes

Lead Engineer

MariaDB Corporation

mmontes11

mmontesperez

@m_montes11

mmontes11

whoami

Creator & Lead Maintainer

mariadb-operator • Kubernetes operator for MariaDB, built with Go and controller-runtime.

Specialized in **Kubernetes • Go • Databases**

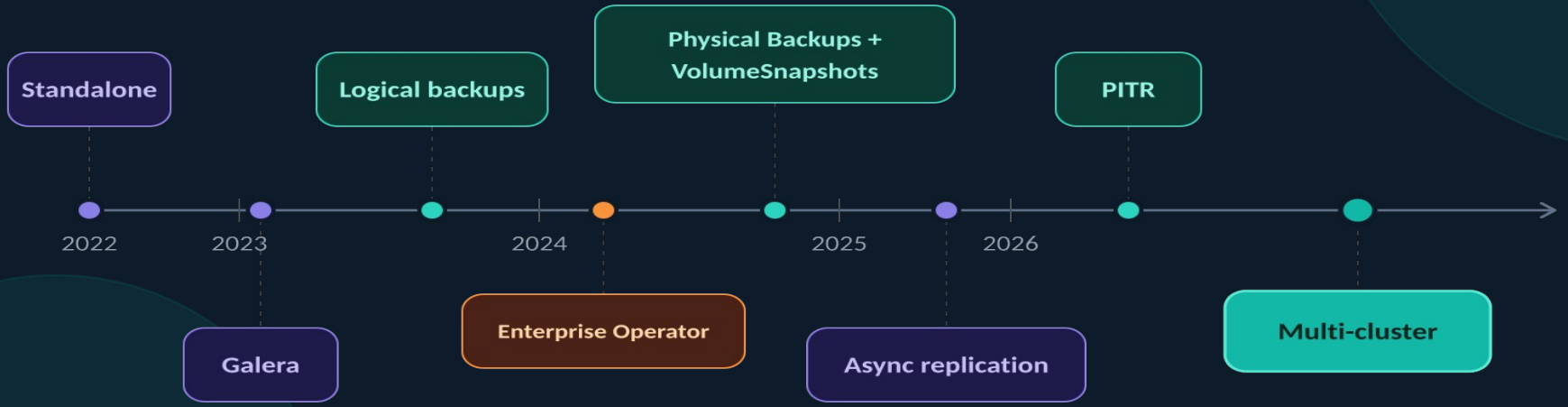
Maintains the operator, ships features, grows the community.

Cloud-native **Open Source** **Cloud-agnostic**

“Run and operate MariaDB in a cloud native way, declaratively manage your database using Kubernetes CRDs rather than imperative commands.”

Project Timeline

From standalone MariaDB to multi-cluster topology in 4 years



Community & Adoption

mariadb-operator · github.com/mariadb-operator/mariadb-operator

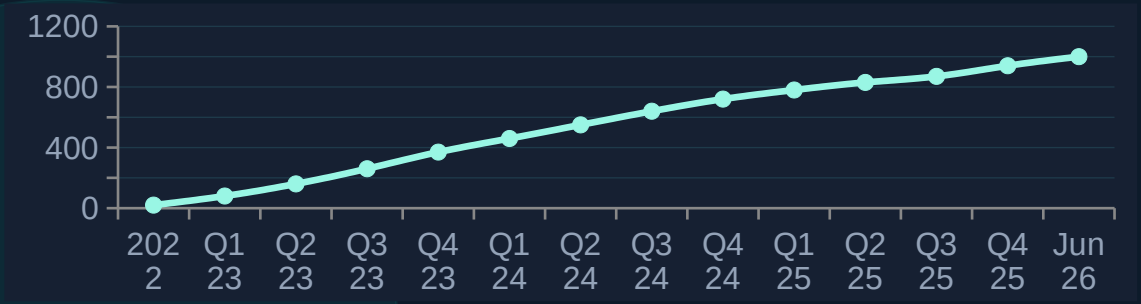
~1K
GitHub Stars

80+
Contributors

1K+
Pull requests

8M+
Image Pulls

★ GitHub Stars over time



157+
Releases

v26.06
Latest

2022
Since

DAY 1

Cluster Setup

Provision a production-ready MariaDB cluster declaratively.

DAY 1 · 1 / 5

Topology

Choose your HA strategy

- **Standalone:** single replica
- **Replication:** asynchronous replication (recommended)
- **Galera:** synchronous multi-master, alternative HA topology
- **Multi-cluster:** cross-cluster replication for multi-region deployments and blue-green upgrades
- All topologies share the same MariaDB CRD, just flip feature flags

```yaml
# Standalone: single node
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  replicas: 1
---
# Replication: semi-sync, auto failover
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  replicas: 3
  replication:
    enabled: true
    semiSyncEnabled: true
    primary:
      autoFailover: true
---
# Galera: synchronous multi-master
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  replicas: 3
  galera:
    enabled: true
---
# Multi-cluster: cross-cluster replication
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  replicas: 3
  replication:
    enabled: true
  multiCluster:
    enabled: true
    primary: mariadb-eu-south
    members:
      - name: mariadb-eu-south
      - name: mariadb-eu-central
```

DAY 1 · 2 / 5

Storage

Persistent volumes for your data

- **StorageClass:** defaults to cluster default, you can use your own
- **Local storage:** strongly recommended. topolvm or local-path-provisioner avoids network I/O latency
- **Block storage:** high IOPs preferred when local isn't an option e.g. AWS io2
- **Volume resize:** update size at runtime, operator handles the expansion (StorageClass should allow)

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  storage:
    # Choose any StorageClass in your cluster.
    # Local storage (e.g. topolvm) avoids
    # network latency.
    storageClassName: topolvm
    size: 100Gi

    # Grow the volume at runtime: bump size and
    # the operator handles the resize.
    # StorageClass must have:
    #   allowVolumeExpansion: true
    resizeInUseVolumes: true
    waitForVolumeResize: true
```

DAY 1 · 3 / 5

Compute

Right-size CPU & memory for MariaDB

- **Requests = Limits:** set both identical, Kubernetes assigns Guaranteed QoS
- **innodb_buffer_pool_size:** set to 70–80% of memory limit for best performance
- myCnf inlined in the CR, operator mounts it automatically

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  resources:
    requests:
      cpu: "4"
      memory: 8Gi
    limits:
      cpu: "4"
      memory: 8Gi

  # innodb_buffer_pool_size should be
  # 70-80% of the memory limit.
  myCnf: |
    [mariadb]
    innodb_buffer_pool_size=6400M
```

DAY 1 · 4 / 5

Scheduling

Policies to implement true high availability

- **Anti-affinity:** Spreads replicas across different nodes
- **Node selector:** select the right instance type (high IOPs, 10Gbps+) and pin pods to dedicated DB nodes
- **Topology spread:** distribute evenly across availability zones
- **Tolerations:** combine with node taints to dedicate nodes exclusively to databases
- **priorityClassName:** system-node-critical, prevents eviction under node pressure

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  replicas: 3
  # Spread each replica on a different node.
  affinity:
    antiAffinityEnabled: true
  # Pin Pods to DB-dedicated nodes.
  nodeSelector:
    node.kubernetes.io/instance-type: m6in.xlarge
  # Spread across availability zones.
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
  # Tolerate the taint on dedicated DB nodes.
  tolerations:
    - key: "mariadb.com/dedicated"
      operator: "Exists"
      effect: "NoSchedule"
  # Limit voluntary disruptions.
  podDisruptionBudget:
    maxUnavailable: "1"
  # Prevent eviction under node pressure.
  priorityClassName: system-node-critical
```

DAY 1 · 5 / 5

TLS

Encrypted connections by default

- **tls.enabled:** operator auto-issues CA + server + client certs
- **tls.required:** reject all non-TLS connections for production
- **cert-manager:** delegate issuance to a ClusterIssuer if preferred
- **Bring your own CA:** provide the CA keypair in a Secret
- Auto-renewal: operator rotates certs before expiry without downtime

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  tls:
    enabled: true
    # Reject any non-TLS connection attempt.
    required: true

    # Option A (default): operator issues and
    # rotates certificates automatically.

    # Option B: delegate issuance to cert-manager.
    serverCertIssuerRef:
      name: root-ca
      kind: ClusterIssuer
    clientCertIssuerRef:
      name: root-ca
      kind: ClusterIssuer

    # Option C: bring your own CA stored
    # in a Kubernetes Secret.
    serverCASecretRef:
      name: mariadb-server-ca
    clientCASecretRef:
      name: mariadb-client-ca
```

DAY 2

Operations

Backup, restoration and updates: all declarative.

DAY 2 · 1 / 4

Physical Backups

Consistent full backups

- **mariadb-backup:** MariaDB-native, hot backup, zero downtime.
- **VolumeSnapshots:** Kubernetes-native, near-instant, CSI-driver-based
- **Storage:** S3-compatible, Azure Blob, PVC, or Kubernetes volumes
- **Scheduling:** cron expression or on-demand trigger
- **Retention:** maxRetention auto-cleans old backups (e.g. 720h = 30 days)

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: PhysicalBackup
metadata:
  name: physicalbackup
spec:
  mariaDbRef:
    name: mariadb
  schedule:
    cron: "0 2 * * *"      # daily at 02:00
    # Update identifier to trigger on-demand:
    onDemand: "1"
  target: Replica          # zero impact on primary
  compression: bzip2
  maxRetention: 720h       # 30 days
  storage:
    # S3-compatible (AWS S3, MinIO, GCS, …)
    s3:
      bucket: physicalbackups
      prefix: mariadb
      endpoint: s3.amazonaws.com
      region: us-east-1
      accessKeyIdSecretKeyRef:
        name: s3-credentials
        key: access-key-id
      secretAccessKeySecretKeyRef:
        name: s3-credentials
        key: secret-access-key
    # azureBlob: { … }      # Azure Blob Storage
    # volumeSnapshot: { … } # consistent, near-instant
```

DAY 2 · 2 / 4

Point-In-Time Recovery

Restore to a given point-in-time

- **Binary log archival:** agent sidecar continuously streams binlogs to S3
- **Base backup + binlogs:** restore physical backup then replay logs
- **targetRecoveryTime:** RFC3339 timestamp, operator finds the right backup
- **Last recoverable time:** visible in PointInTimeRecovery status, drives RPO

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: PointInTimeRecovery
metadata:
  name: pitr
spec:
  physicalBackupRef:
    name: physicalbackup
  archiveTimeout: 5m
  storage:
    s3:
      bucket: binlogs
      endpoint: s3.amazonaws.com
      accessKeyIdSecretKeyRef:
        name: s3-credentials
        key: access-key-id
      secretAccessKeySecretKeyRef:
        name: s3-credentials
        key: secret-access-key
---
# Restore to a specific point in time:
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  bootstrapFrom:
    pointInTimeRecoveryRef:
      name: pitr
    targetRecoveryTime: "2025-06-18T14:37:52Z"
```

DAY 2 · 3 / 4

Restoration

Bootstrap new clusters from existing data

- **bootstrapFrom:** declare the restore source directly in the MariaDB CR
- **PhysicalBackup ref:** restore from the closest backup before targetRecoveryTime
- **PITR:** combine physical backup + binlog replay for point-in-time restoration
- Staging storage PVC avoids exhausting node disk for large backups

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  replicas: 3
  replication:
    enabled: true

  bootstrapFrom:
    # A) Restore closest PhysicalBackup.
    backupRef:
      name: physicalbackup
      kind: PhysicalBackup

    # B) PITR — restore base backup then
    # replay binlogs up to targetRecoveryTime.
    pointInTimeRecoveryRef:
      name: pitr
    targetRecoveryTime: "2025-06-18T14:37:52Z"

    stagingStorage:
      persistentVolumeClaim:
        resources:
          requests:
            storage: 100Gi
        accessModes:
          ReadWriteOnce
```

DAY 2 · 4 / 4

Updates

Zero-downtime rolling upgrades

- **ReplicasFirstPrimaryLast:** roll replicas one-by-one, primary last (default strategy)
- **Never:** freeze updates, useful during maintenance windows or staged rollouts
- **OnDelete:** full manual control, delete a pod to trigger its update
- **autoUpdateDataPlane:** automatically updates operator init container and sidecar images to match the operator version

```yaml
# Default rolling strategy: replicas first,
# then the primary.
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  updateStrategy:
    type: ReplicasFirstPrimaryLast
    autoUpdateDataPlane: true
---
# Pause all automatic image updates during
# a maintenance freeze or staged rollout.
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  updateStrategy:
    type: Never
---
# Full manual control: delete a Pod to
# trigger its individual update.
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  updateStrategy:
    type: OnDelete
```

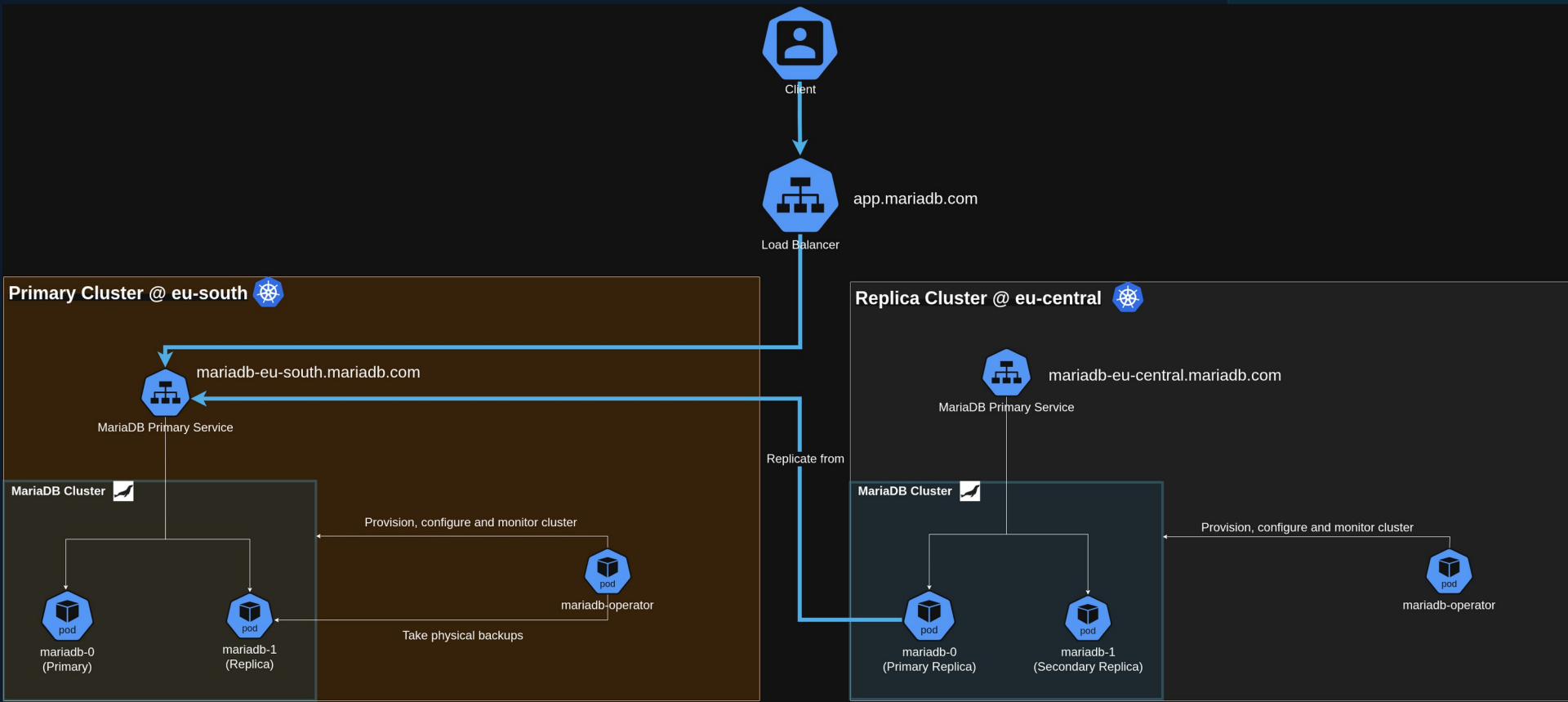
DEMO

Multi-Cluster

Cross-cluster replication · Multi-region HA · Blue-green upgrades

Architecture

Cross-region deployment using the multi-cluster topology



Thank you!

Run and Operate MariaDB in a Cloud Native Way

Useful links

Documentation	`github.com/mariadb-operator/mariadb-operator/blob/main/docs/README.md`
Demo	`github.com/mariadb-operator/mariadb-operator/blob/main/demos/multi-cluster`
GitHub repo	`github.com/mariadb-operator/mariadb-operator`

Martin Montes · Lead Engineer, MariaDB Corporation

Scan to open repo